

Real-Time Yoga Pose Classification and Feedback Using Joint Angles and MediaPipe

Simay Pay Mikiko Nomiya Elif Ayhan

Keywords: Human Pose Estimation and Correction, MediaPipe, Machine Learning, Real-Time Feedback

Abstract: As the trend of practicing yoga at home rises, the need for a system that could evaluate posture accuracy with feedback is also increasing. To fully benefit from a yoga practice, maintaining correct posture is crucial. Hence, this project aims to develop a real-time system for classifying yoga poses and providing feedback, using joint-angle calculations along with pose estimation via MediaPipe. The system uses a webcam and the MediaPipe framework to derive body landmarks from video input. From these landmarks, 13 relevant joint angles are computed using trigonometry and are collected. These angles are then used as input for a KNN classifier which is trained on a custom dataset of 8 unique yoga positions. After detecting the corresponding yoga pose, the system features a feedback mechanism that indicates posture accuracy on screen by comparing the user's joint angles with an ideal reference threshold.

1 INTRODUCTION

Over the past few decades, yoga has experienced significant growth in popularity. Once a specialized wellness discipline, its accessibility and numerous health benefits have helped it become a mainstream form of exercise. The most important component of yoga, however, is proper pose alignment; when done incorrectly, the practice loses many of its benefits and could lead to long-term posture issues and muscle damage. Standardized, monitored environments enable instructors to adjust practitioners' postures in real-time. Still, the trend of unsupervised yoga at home, makes a self-automated system with corrective posture input increasingly necessary. Pose Estimation, a computer vision challenge that aims to detect the location and orientation of a person or object, makes this technology possible. Typically, in the context of Human Pose Estimation, this is achieved by projecting the locations of specific critical points (such as hands, head, elbows, etc.) within the context. Yoga positions mainly depend on the exact alignment of joints hence, exact tracking of those joints is crucial. Pose Estimation provides a reliable camera-based solution for this situation.

From CNN-based deep learning on raw images (Sharma et al., 2022) to more classical machine

learning algorithms with joint angle classification (Dutta et al., 2021; Joshi & Prasad, 2021), several techniques have been used for yoga posture discovery. While deep learning approaches that utilize convolutional neural networks (CNNs), such as those in Sharma et al. (2022), have achieved excellent accuracy without the need for handcrafted features, the model also requires a larger dataset and does not enable posture correction. Other methods, which emphasize joint angle classification (Dutta et al., 2021; Joshi & Prasad, 2021), employ a lightweight technique for both correction and actual classification. Still, their classification stayed black-boxed with little interpretability.

Our method avoids the significant computational demands while concentrating on a simpler, more interpretable answer, thereby building on these roots. This project aims to develop a real-time system that utilizes camera input to classify yoga poses and provide feedback. Using joint angles obtained via MediaPipe posture estimation, this system will precisely identify different yoga poses and provide joint-level modifications through visual feedback. This joint angle classification with Mediapipe focuses solely on body geometry, unlike other CNN-based methods. With this method, visual distractions—

which may arise with pixel-based classification are eliminated. This simplifies the comparison since the angles can be immediately compared in quantified degrees. With fewer datasets, this categorization is more effective. Especially in light of other methods that rely on PoseNet (Joshi & Prasad, 2021), which only offer 2D landmarks, the utilization of the Mediapipe architecture also has considerable value. Mediapipe extracts the coordinates of important body joints (such as elbows, knees, and shoulders) from a webcam feed to provide more consistent landmark identification in 3D. The system processes this input in a pipeline as shown in **Figure 1** and discussed further in **Section 2.3**. This report will be further structured into Method, Results and References sections.

2 METHOD

The system takes webcam video input, finds and analyzes the user's body landmarks using Mediapipe, calculates the critical joint angles, and classifies the yoga pose with a trained KNN model. After classification based on the predicted pose, the system compares the user's angle outputs with the predefined ideal angles and given thresholds. Visual feedback is displayed directly on the user's body through emoji icons corresponding to the threshold location of the values, indicating whether the joint is in the correct position, at proximity, or severely misaligned. This modular pipeline is composed of these primary stages:

Pose Detection: With MediaPipe, the system detects 33 body landmarks from video frames.

Feature Extraction: From these landmarks, a subset of 13 joint angles is determined using a trigonometric function.

Pose Classification: Based on the angle values, a KNN classifier, which is trained on a dataset of labeled poses, predicts the current pose.

Pose Feedback: The system compares the current and the ideal angle values hence, provides real-time feedback using virtual emoji indicators for each joint.

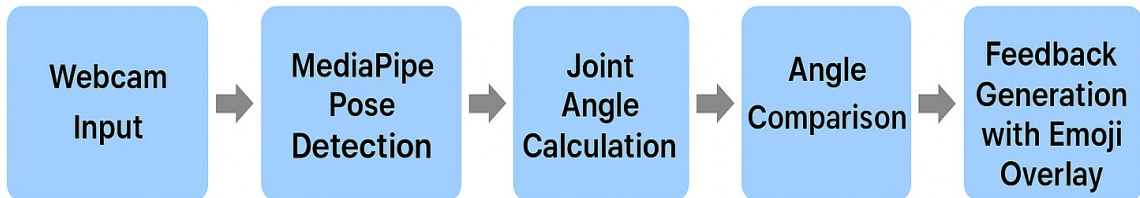


Figure 1. The detailed flowchart of the yoga classification and feedback pipeline

2.1 Dataset Construction

To train a machine learning model that classifies yoga poses, a custom dataset was needed. Our system focuses on extracting joint angles as a pose-specific feature rather than directly using raw images for classification. The dataset was collected using multiple labeled folders, where each folder contains photos of users performing a specific pose, and the folders are named accordingly. There are 8 primary poses, 4 of which are labeled separately for both the right and left sides to provide better clarity during training. The Mediapipe Framework is used to process



Figure 2. Landmarks detected by Mediapipe

each image by estimating 33 body landmarks with 3D (x, y, z) coordinates to represent joint positions (e.g., **Figure 2**). From the 33 landmarks, 3 were selected for each of the 13 joints to compute the necessary angles (left knee, right knee, right shoulder, etc.). The x and y coordinates of the landmarks are put through a custom trigonometric function in order to calculate the angle, based on the arctangent vector differences. As shown in **Figure 3** the final dataset consists of lines representing each image, where each line contains a pose label (e.g., Tree_left) followed by a list of 13 joint angles. This table is then stored as a CSV file called *dataset_images.csv*, with more than 1200 labeled entries.

Pose	Left_Elbow	Right_Elbow	...	Right_Wrist
Tree_Left	125.6	178.2	...	164.5
Warrior_R	167.0	180.1	...	195.3

Figure 3. Sample Rows from *dataset_images.csv*.

2.2 Model Training

Since the dataset that will be used to train the model is made, the model is generalized by partitioning the dataset into training and testing subsets, utilizing 80% of the data for model training and the remaining 20% for evaluating unknown data. We decide on K-Nearest Neighbors (KNN) as our classification technique (Pedregosa et al., 2011) for machine learning. KNN is a simple instance-based learning algorithm that, with given data, calculates the distance between that point and all points in the dataset. The similarity is often measured by Euclidean Distance. Then, it selects the 'k' closest points and determines the class of this data regarding selected neighbors. We have also considered more complex models, such as CNNs, but given the size of our dataset and our joint angle geometric approach, KNN provided a simple, straightforward classification.

To ensure the generalizability of our classification model, we utilized k-fold cross-validation with scikit-learn's `cross_val_score` function and a confusion matrix to demonstrate the model's accuracy in predicting each pose, with diagonal values representing correct classifications and off-diagonal values signifying misclassifications.

2.3 The Real Time Prediction and Feedback System

After training the KNN model, the pipeline shown in **Figure 1** occurs as follows, beginning with webcam input and finishing with real-time feedback. This stage turns the static model into an interactive feedback system.

2.3.1 Live Pose Capture and Estimation

To capture continuous webcam input, OpenCV's `cv2.VideoCapture(0)` and Mediapipe's landmark detection are used. Each captured frame is converted from BGR (OpenCV) to RGB format, which is the input format expected by MediaPipe.

2.3.2 Angle Computation and Pose Prediction

After detecting the landmarks and their coordinates, 13 joint angles are computed with a trigonometric formula (**Figure 4**). These angles form a feature vector, which is then fed into the trained KNN to predict the yoga pose.

$$\theta = \arctan 2(y_3 - y_2, x_3 - x_2) - \arctan 2(y_1 - y_2, x_1 - x_2)$$

$$\theta = \left(\frac{180}{\pi} \times \theta \right) \mod 360$$

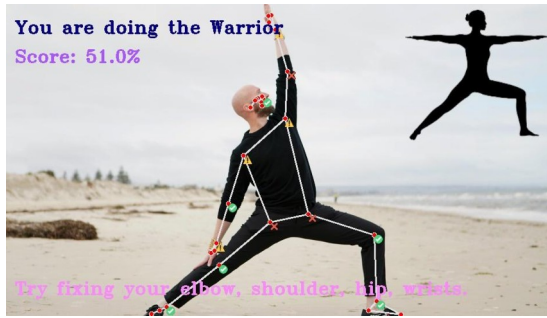
Figure 4. The formula used to compute the angle between three points

2.3.3 Feedback

Depending on the pose, the system compares the user's angles to the ideal reference angles of the corresponding pose and, based on their deviation thresholds, a feedback function aligns each joint with an emoji :

-  Correct (within "good" threshold)
-  Warning (within "warn" threshold)
-  Incorrect (outside all thresholds)

On-screen, the user sees each joint marked with a corresponding emoji, the predicted pose label, a visual reference of the pose and the overall score, which is a weighted computation of correct and incorrect positions. Finally, a motivational text is displayed regarding this score ("PERFECT!", "Nice!", etc.). If the total score of a user is below 70% , there will be an extra custom message with a list of joints that needs



correction. More explanation about the calculation of the score and the text messages will further be discussed on **Section 3.3** . To ensure optimization, this feedback is updated every 20 frames to avoid sudden glitches and provide a smoother application. Landmark visibility is verified before feedback presentation, guaranteeing that only visible joints are assessed. Predictions and feedback from prior frames are stored to ensure display consistency. This feedback transforms the system from a classifier into an actual posture corrector.



Figure 5. A side to side visual on-screen display of two user performances with different levels of posture correctness.

3 RESULTS

3.1 Model Performance Overview

As briefly mentioned earlier, in the method for calculating the accuracy and performance of our model, the dataset is split into two subsets. The final dataset consists of around 1200 labeled samples, with 80% (960 photos) allocated for training and the remaining 20% (240 images) designated for testing. The 80/20 division offers an optimal equilibrium for our dataset, guaranteeing sample data for learning and evaluation purposes.

The model employs a KNN classifier with $k = 3$, which evaluates the current data against its three nearest neighbors based on distance. Establishing $k = 3$ achieves an optimal equilibrium among values, as a diminished k (e.g., 1) renders the categorization excessively sensitive, while an augmented k (e.g., 7) neglects and diminishes essential data.

Alongside a singular train/test split, we furthermore utilized cross-validation through `cross_val_score()` from scikit-learn. This approach partitions the dataset into five segments, utilizing four segments for model training and reserving the fifth segment for testing. By computing the accuracy scores across all folds, we obtained a more reliable performance metric that reduces the likelihood of overfitting.

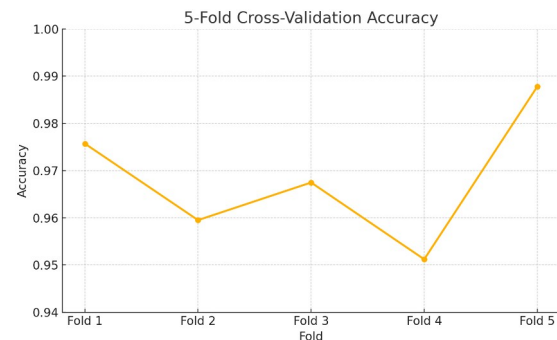
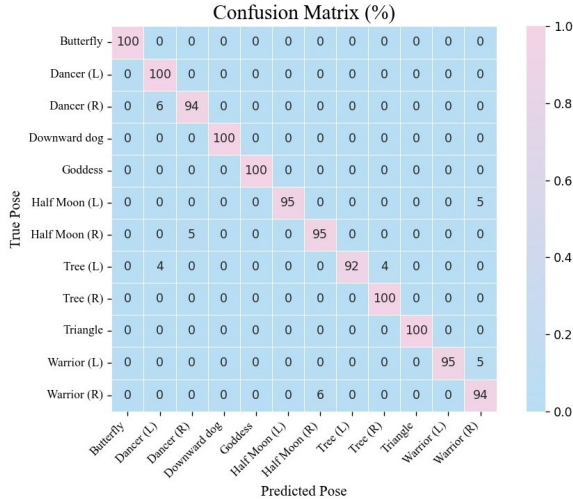


Figure 6. Cross-validation accuracy scores across 5 folds demonstrating the model's generalization performance

As shown in **Figure 6**, the accuracy scores of folds stayed consistent and above 95% among each fold. Our model attained a 95.5% test accuracy on the held out 20% of the dataset indicating its effective performance with the provided dataset.

3.2 Confusion Matrix Analysis

We employed a confusion matrix to illustrate the model's efficacy in identifying each yoga posture. A confusion matrix is a N x N table, with N being the number of classes, namely yoga poses in this instance. Each row represents the actual class, while each column indicates the predicted class. The accuracy score produces a singular result, while a confusion matrix offers predictions for each class individually.



Poses like "Tree_right," "Butterfly" and "Goddess" had nearly flawless categorization, evidenced by strong diagonal counts and minimal misclassification. Some positions exhibited similarity due to visual or structural likeness: "Dancer_right" was confused by "Dancer_left" (6%) and "Half Moon_left" was confused by "Warrior_right" (5%).

There is no major missclassification above 10% and most of the confusion occurred between left and right versions of the poses which is expected due to mirror symmetry. Other minor misclassifications often stemmed from subtle differences in joint angles, especially in shoulder and hip orientation, which can be visually deceptive even to the human observer.

3.3 Feedback System Performance

The feedback system is the core component of the project when it comes to posture correction. It extracts angles from the user and compares them to the ideal angles of the corresponding pose. The output of this process is a real-time evaluation using emojis (✅, ⚠️, ❌) which are placed directly on the respective joint, accompanied by a total percentage.

To evaluate a performance, we have implemented a weighted joint-based total correctness and accuracy score. Every joint has a unique weight, specific to the

pose. The emoji labels are also converted to values. The formulation is shown as follows:

$$\text{Feedback Score} = \sum_{i=1}^n f_i \cdot w_i$$

f_i = a numerical feedback value for joint i (1.0(✅), 0.3(⚠️), or 0.0(❌)),

w_i = weight (%) given to joint i for the specific pose,

n = number of joints (13 in this system).

Given that the weights add up to 100, the final score inherently falls within the range of 0 to 100.

- A score above 85% corresponds to correct posture with a special text message "PERFECT!".
- A score between 85-70% corresponds to good alignment with a special text message "Nice"
- A score below 70% indicates severe misalignment with a special message of a list of joints that needs correction.

Example feedback visuals can be seen in **Figure 5**

3.4 Summary of Challenges

Our model utilizes Mediapipe pose detection to identify the body's landmarks. However, the output we retained from the Mediapipe library is the coordinates (x, y, z, vis). Training the model with the coordinates of the landmarks could have introduced issues with scaling and centering. We had to take into account that each user would have different limb lengths, torso sizes, and so on. We would also have to center the coordinates to classify each pose. Moreover, in our feedback system, we aim to display a user's performance by adding an indicator to each joint. We decided to use classification not by coordinates but by angles between landmarks, also known as joints. This way, scaling was not an issue. We have selected 13 essential joints and calculated the angles for each using a function that determines the angle between three landmarks based on their x and y coordinates. The model easily classifies each pose based on its distinct angles.

Performing a prediction per frame introduced a slight performance lag and was likely to cause glitches. Therefore, to optimize performance, we opted for an

approach that updates the predictions once every 20 frames and caches them for a smoother application rather than a frame-by-frame approach.

Some of the poses in yoga are not symmetrical. However, our model can classify the pose because it learns discriminative patterns rather than relying on global body symmetry. Yet, we encountered an issue with our feedback system. Let's take an example from the "tree" pose. We could not give feedback on exactly which leg to lift because our feedback system works with hardcoded ideal angles for each pose we are considering.

We were either going to add conditions to our feedback system or change the training. Since we had already created our own CSV files for training from the dataset, all we needed to do was find a new dataset with separate labels for right and left and retrain our model.

3.5 Future Improvements

Although the current visual emoji feedback is sufficient to correct posture, more advanced and detailed feedback would make the process easier for

users to improve their posture. The current emoji feedback indicates how accurately you are placing each joint but not necessarily how to improve it (e.g., the right elbow is mispositioned, but the user doesn't know whether to raise or lower it). Perhaps adding an extra emoji with side arrows (➡️, ⬆️, ⬇️, ⬅️, etc.) to indicate the direction of movement for the joints would be beneficial.

User-specific calibration would help accommodate people's different body proportions, especially their flexibility, by allowing the system to adapt to custom angle ranges.

When the user feedback system is optimized, it can be integrated to allow multi-person tracking, which can even be used in supervised yoga sessions as a detection system.

Currently to get a meaning feedback, the user has to stand in front of the camera with the right angle because our system relies on 2D joint positions for feedback. Adding a 3D pose estimation would make the feedback more accurate and, it would allow a larger range of movements. This change would also let us to add poses that involve depth-based movements (e.g. forward bends).

4 REFERENCES

- Dutta, S., S. Paul, and S. Bhattacharya. 2021. "YogaPoseNet: Real-time Yoga Posture Classification Using Deep Neural Networks." In *Computer Vision and Image Processing*, 284–294. Springer.
- Joshi, A., and D. K. Prasad. 2021. "Yoga Posture Recognition Using CNN and Transfer Learning Techniques." In *Proceedings of the 2021 10th International Conference on System Modeling & Advancement in Research Trends (SMART)*, 264–269. IEEE.
- Sharma, A., G. Singh, and R. Tripathi. 2022. "Real-Time Feedback System for Yoga Posture Correction Using Machine Learning." *International Journal of Interactive Multimedia and Artificial Intelligence* 9 (5): 50–58.
- Lugaresi, C., et al. 2019. "MediaPipe: A Framework for Building Perception Pipelines." *arXiv preprint arXiv:1906.08172*.
- Cao, Z., G. Hidalgo, T. Simon, S. E. Wei, and Y. Sheikh. 2017. "Realtime Multi-person 2D Pose Estimation Using Part Affinity Fields." In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 7291–7299.
- Pedregosa, F., et al. 2011. "Scikit-learn: Machine Learning in Python." *Journal of Machine Learning Research* 12: 2825–2830.
- Hunter, J. D. 2007. "Matplotlib: A 2D Graphics Environment." *Computing in Science & Engineering* 9 (3): 90–95.
- Waskom, M. L. 2021. "Seaborn: Statistical Data Visualization." *Journal of Open Source Software* 6 (60): 3021.

